

Incident RCA: Tasker Pod CrashLoop (INC-001)

Metadata	Details
Incident ID	INC-001
Service	Tasker Web API (tasker-tasker-chart)
Environment	Kubernetes (Namespace: default)
Severity	Critical
Status	Resolved
Root Cause	Missing / Empty Database Host Configuration (DB_HOST="")
Detection Alert	TaskerPodCrashLooping (Prometheus)
Simulation Script	tasker/tests/simulate-crashloop.sh

1. Executive Summary

A configuration defect was injected into the Tasker deployment by setting the database host (`DB_HOST`) in `ConfigMap/tasker-tasker-chart-config` to an empty string (`""`). Upon rolling restart, the Flask application (running under Gunicorn) failed during bootstrap while executing `db.create_all()` in `app.py`. With `DB_HOST` empty, `psycopg2/libpq` fell back to a local Unix domain socket path (`/var/run/postgresql/.s.PGSQL.5432`), which does not exist in the Tasker container (PostgreSQL runs in a separate pod, not locally).

The worker process raised `psycopg2.OperationalError` and exited immediately, causing Gunicorn's master process to shut down. Kubelet restarted the container repeatedly, and the pod entered `CrashLoopBackOff`. Prometheus detected the sustained failure and raised the `TaskerPodCrashLooping` alert. The configuration was restored to `tasker-postgresql` and the deployment restarted, returning the pod to `1/1 Running`.

Incident in one sentence: We intentionally set Tasker's `DB_HOST` to an empty value, causing the application to attempt a nonexistent local PostgreSQL socket instead of reaching the real database pod, which made the container repeatedly fail during startup and enter `CrashLoopBackOff`; Prometheus detected it, we confirmed the cause from application logs, restored `DB_HOST=tasker-postgresql`, and verified the pod recovered.

2. Architecture & Baseline Context

Tasker Pod (default ns)	PostgreSQL Pod (default ns)
+-----+	+-----+
Gunicorn / Flask TCP Service: tasker-postgresql	
app.py / models.py :5432-> Port: 5432	
(no local pg socket)	
+-----+	+-----+

- **Application:** Flask REST service using SQLAlchemy and `psycopg2`.
- **Intended Connection:** `postgresql://postgres:<password>@tasker-postgresql:5432/tasker`
- **Config Provider:** ConfigMap `tasker-tasker-chart-config`, mapped via environment variables.

3. Incident Timeline

Note: This RCA documents a verification re-run of the CrashLoop simulation, executed on 2026-09-04 to capture exact application logs for this document. The fault mechanism, alert behavior, and recovery procedure are identical to the original test run performed earlier in the exercise; only this run's timestamps and log output are cited directly.

Timestamp (UTC)	Phase	Action / Event
~11:41:5x	Trigger	<code>DB_HOST</code> patched to <code>""</code> in <code>tasker-tasker-chart-config</code> ; deployment rollout restarted.
11:42:01	Boot	New pod (<code>tasker-tasker-chart-767bd45c98-1hjn4</code>) started; Gunicorn booted worker (pid 7).
11:42:03	Failure	Worker raised <code>psycopg2.OperationalError</code> on local Unix socket lookup; worker exited (code

		3); Unicorn master shut down.
11:42:03+	Escalation	Kubelet restarted the container; pod cycled to <code>CrashLoopBackOff</code> (confirmed 4 restarts within ~3 minutes).
~11:44-11:47	Detection	<code>TaskerPodCrashLooping</code> fired after the alert's <code>for: 2m</code> evaluation window.
17:12	Investigation	Confirmed via <code>kubectl logs</code> that the crashing pod's error matched the expected Unix-socket-fallback failure.
17:12	Mitigation	<code>DB_HOST</code> patched back to <code>tasker-postgresql</code> ; deployment restarted.
17:13	Recovery	New pod (<code>tasker-tasker-chart-79df9b776d-2fgvp</code>) reached 1/1 Running, 0 restarts. Old pods terminated cleanly.

4. Trigger & Execution Mechanism

```
kubectl patch configmap tasker-tasker-chart-config -n default \
  --type merge \
  -p '{"data":{"DB_HOST":""}}'

kubectl rollout restart deployment/tasker-tasker-chart -n default
```

Technical cause in application logic (`app.py`):

```
DB_HOST = os.environ["DB_HOST"]
app.config["SQLALCHEMY_DATABASE_URI"] = (
    f"postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
)
db.init_app(app)

with app.app_context():
    db.create_all() # Executed synchronously during module import
```

With `DB_HOST=""`, the resulting URI omits a hostname before the colon. In `libpq/psycopg2`, this triggers Unix domain socket mode rather than TCP, targeting `/var/run/postgresql/.s.PGSQL.5432`. Because PostgreSQL runs in a separate pod (not locally), no such socket exists.

5. Detection & Alerting Verification

```
- alert: TaskerPodCrashLooping
  expr: rate(kube_pod_container_status_restarts_total{container="tasker"}[5m]) * 60 > 0
        or kube_pod_container_status_waiting_reason{reason="CrashLoopBackOff"} == 1
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "Tasker pod is crash-looping"
    description: "Tasker container in pod {{ $labels.pod }} is repeatedly failing and restarting."
```

Confirmed via `kubectl get pods -n default`, which showed `tasker-tasker-chart-767bd45c98-1hjn4` cycling through `CrashLoopBackOff` with restart count incrementing (4 restarts in ~3 minutes).

6. Investigation & Diagnostics

Log extraction (verbatim, captured directly from the crashing pod):

```
kubectl logs -n default tasker-tasker-chart-767bd45c98-1hjn4
```

Key output:

```
psycopg2.OperationalError: connection to server on socket
"/var/run/postgresql/.s.PGSQL.5432" failed: No such file or directory
Is the server running locally and accepting connections on that socket?
...
[2026-09-04 11:42:03 +0000] [7] [INFO] Worker exiting (pid: 7)
[2026-09-04 11:42:03 +0000] [1] [ERROR] Worker (pid:7) exited with code 3.
[2026-09-04 11:42:03 +0000] [1] [ERROR] Shutting down: Master
```

```
[2026-09-04 11:42:03 +0000] [1] [ERROR] Reason: Worker failed to boot.
```

This directly confirms the local-socket-fallback failure mode described in Section 4 – the error is not a generic “connection refused” to a resolved host, but explicitly a missing local socket file, consistent only with an empty/unset host value.

Configuration audit:

```
kubectl get configmap tasker-tasker-chart-config -n default -o yaml
```

Confirmed `DB_HOST`: "" at time of failure.

Database health check:

```
kubectl get pods -l app.kubernetes.io/name=postgresql -n default
```

PostgreSQL pod remained 1/1 Running throughout – confirming this was an application configuration failure, not a database outage.

Caution: An earlier, unrelated log entry (from a node-level `SandboxChanged` event that restarted both pods together) showed a different error – “Connection refused” to a resolved IP – which could be mistaken for this incident’s cause. That entry does **not** relate to this fault; it reflects a normal startup race after an infrastructure-level pod recreation, not the `DB_HOST=""` misconfiguration.

7. Recovery & Verification Steps

```
kubectl patch configmap tasker-tasker-chart-config -n default \
--type merge \
-p '{"data":{"DB_HOST":"tasker-postgresql"}}'

kubectl rollout restart deployment/tasker-tasker-chart -n default
```

Verified:

```
kubectl get pods -n default
tasker-tasker-chart-79df9b776d-2fgvp    1/1    Running    0    19s
tasker-postgresql-0                    1/1    Running    1    5h42m
```

Alert confirmed cleared in Prometheus/Alertmanager once the pod stabilized.

8. Root Cause Analysis (5 Whys)

1. **Why did the Tasker pod enter CrashLoopBackOff?** The application crashed immediately during startup.
2. **Why did the application crash during startup?** `db.create_all()` raised an unhandled `psycopg2.OperationalError`.
3. **Why did psycopg2 fail to connect?** It attempted to connect via a local Unix domain socket (`/var/run/postgresql/.s.PGSQL.5432`) which does not exist in this container.
4. **Why was psycopg2 attempting a local socket connection?** The database connection string had an empty host segment, which libpq interprets as a request for local socket connection rather than TCP.
5. **Why was the host empty?** The ConfigMap `tasker-tasker-chart-config` had `DB_HOST` patched to "" as part of this simulation.

Root cause: No startup-time validation or distinct error signal for missing/invalid required configuration (`DB_HOST`); an empty value is silently accepted and only fails downstream during the DB connection attempt.

9. Preventative & Detective Action Items

Category	Action Item	Target	Owner
Preventative	Add startup validation in <code>app.py</code> to assert <code>DB_HOST</code> is non-empty before initiating database engine setup, with a clear distinct error message.	Application	Dev
Preventative	Implement Helm chart schema validation (<code>values.schema.json</code>) to disallow empty strings for <code>db.host</code> .	Helm Chart	DevOps

Detective	Maintain the 2-minute <code>TaskerPodCrashLooping</code> alert threshold as proven effective during this test.	Monitoring	SRE
Operational	Document the recovery patch in SRE runbooks for rapid rollback of erroneous ConfigMap updates.	Runbooks	SRE